

Progressive Autonomy as Preference Learning

A Formalization of Trust Calibration for Agentic Tool Use

Changkun Ou

Latere AI, Munich, Germany

September 28, 2026 (version 3)

Abstract

We formalize trust calibration for agentic tool use (deciding whether an agent’s proposed action may run autonomously, must be escalated to a human, or is blocked) as Gaussian-process classification with a reject option, learned online from the supervisor’s approve/deny feedback. The three-tier gateway is the Bayes-optimal one-step decision under Chow’s reject-option loss, with thresholds $\tau_{\text{high}} = 1 - c_{\text{ask}}/c_{\text{FA}}$ and $\tau_{\text{low}} = c_{\text{ask}}/c_{\text{FB}}$ set by stated costs rather than tuned. We prove that unary approve/deny feedback identifies the allow/deny boundary while pairwise (preferential) feedback cannot; that a separable time-decaying kernel forgets static action risk and forces every unlabelled action back into escalation; that the irreducible escalation rate is a property of the supervisor; and that random audits give unbiased false-allow estimates, with $n \geq \ln(1/\delta)/\alpha$ clean audits certifying a rate below α . In simulation, an additive kernel that separates static risk from drifting tolerance halves human labels per action (28.4% $\rightarrow$ 13.5%) and cuts cost-weighted regret by 59% at an unchanged false-allow rate. We then replace the tool taxonomy with an opaque LLM judge, as in shell-centric agent harnesses: a judge’s error rate for a supervisor is not identifiable from its verdicts, changes when the supervisor’s tolerance changes, and is estimable without bias only from labels with known propensities. In a pre-registered simulation the calibrated gateway matches the regret of escalating the judge’s blocks with half the human labels, but cannot repair a judge blind spot its features do not expose. The cores of all eight propositions are machine-checked in Lean 4 with Mathlib. The problem is related to, but not an instance of, preferential Bayesian optimization.

1 Introduction and Related Work

Coding agents today gate tool calls with a binary switch: run automatically, or refuse. A refusal does not pause the run for a human; the agent receives it and keeps going, often routing around the obstacle in ways that are worse than asking would have been. The missing primitive is not *block* but *escalate*, and the point at which to escalate should be learned from the supervisor’s own approve/deny history rather than written by hand. This paper formalizes that learning problem and analyses the decision rule, the model of drift, and the evidence the supervisor’s labels can and cannot provide.

Contributions. (i) We show that the three-tier ALLOW/ASK/BLOCK gateway is exactly Chow’s reject-option rule [12], so its thresholds follow in closed form from costs and it depends on the posterior only through the mean approval probability (Proposition 2); the long-run escalation rate is bounded below by the supervisor’s own ambiguity (Proposition 3). (ii) We show that unary feedback identifies the allow/deny boundary while pairwise feedback cannot (Proposition 1), which

locates the problem precisely relative to preferential Bayesian optimization. (iii) We prove that a separable time-decaying kernel forgets static risk and forces re-escalation after at most $\lambda \ln(m/z)$ label-free steps (Proposition 4), and propose an additive decomposition $f(x, t) = \tau(t) - r(x)$ that forgets only what drifts. (iv) We treat the selective-labels structure of the gateway, where auto-decided actions are never labelled, and show that random audits restore unbiased error estimates and give a certification sample size (Proposition 5). (v) We extend the gateway to arbitrary shell actions judged by an opaque LLM, formalize the two questions such a judge raises (is it trustworthy, and how likely is a verdict to be right) and show what can and cannot be learned about them (Propositions 6–8). (vi) The cores of all eight propositions are machine-checked in Lean 4 (Section 13). (vii) Simulation studies measure each claim and correct several claims of the first version of this paper (Appendix B).

Selective classification and deferral. A classifier that may abstain at a cost is Chow’s reject option [12], developed as selective classification [20, 22]; learning when to hand a decision to a human expert is learning to defer [35, 38]. Our ASK tier is the reject option, with the twist that the human’s answer on an abstained action is also the only source of training labels, the setting of online selective classification with limited feedback [21].

Gaussian-process classification and preference learning. The inference is classical GP classification: a GP prior, a probit likelihood and an approximate posterior by the Laplace method or expectation propagation [36, 42]. Chu and Ghahramani [13] introduced GP preference learning with a pairwise probit likelihood, and González et al. [24] embedded preference learning in the query loop of Bayesian optimization [9, 47], placing a GP on the space of duels; Benavoli and Azzimonti [5] survey GP preference and choice models. Our observation model is the unary special case of this family, but Section 9 shows that the difference from pairwise feedback is one of identifiability, not a degenerate special case.

Active learning and level-set estimation. Choosing which actions to send to the human is stream-based selective sampling [14, 46]: the agent, not the learner, chooses the action, and the learner only decides whether to ask. Learning the region where f exceeds a threshold is level-set estimation [8, 25], studied with Bernoulli outcomes by Letham et al. [34]; information-theoretic acquisition for GP classification is BALD [27], and uncertainty sampling is known to suffer from sampling bias [15, 16].

Non-stationarity. A supervisor’s tolerance drifts. Time-varying GP bandits model drift with a forgetting kernel [7], of which our k_{time} is the continuous-time analogue; non-stationary covariances [40] and online changepoint detection [1] are alternatives. Additive kernels [19] let a model forget some components and not others.

LLM judges and monitors. Language-model judges now gate agent actions in deployed coding agents [28]. Calibrated selective evaluation with an agreement guarantee [29], prediction-powered and active inference from few human labels [4, 52], latent-class models of annotator error [2, 17], and AI control with trusted monitors and audit budgets [6, 26] all bear on how far such a judge can be trusted; judges’ errors are correlated across models [23, 30] and they can be attacked [43].

Trust in automation and agent governance. Appropriate reliance aligns trust with trustworthiness [33], and trust calibration is longitudinal [18]. Graduated autonomy is an explicit

deployment axis for language-model agents [37]; recent work catalogues their risks [10], argues for visibility and oversight [11] and for human approval of consequential actions [48]. These accounts leave the escalation policy as a fixed tier; our contribution is a learning rule for it. Because auto-decided actions produce no label, the evaluation of such a rule faces the selective-labels problem [32].

2 Setup

At each decision point $t = 1, 2, \dots$, the gateway observes a proposed agent action $a_t \in \mathcal{A}$ and an execution context $c_t \in \mathcal{C}$,

$$a_t = (\text{tool_name}, \text{args}, \text{target_resource}), \quad (1)$$

$$c_t = (\text{repo_state}, \text{task_desc}, \text{session_history}), \quad (2)$$

and writes $x_t := (a_t, c_t) \in \mathcal{X}$. The agent chooses x_t ; the gateway chooses only a decision $d_t \in \{\text{ALLOW}, \text{ASK}, \text{BLOCK}\}$. If $d_t = \text{ASK}$ the human supervisor returns $y_t \in \{0, 1\}$ (deny/approve), which both settles the action and becomes a training label; ALLOW and BLOCK are executed without a label. Decisions are made prequentially: d_t depends only on labels received before t .

3 Latent Tolerance and Identifiability

Definition 1 (Latent tolerance). *There is a latent function $f: \mathcal{X} \times \mathbb{R} \rightarrow \mathbb{R}$ such that the supervisor approves x at time t with probability*

$$\Pr(y = 1 \mid x, t) = \Phi(f(x, t)), \quad f(x, t) = \tau(t) - r(x) + h(x, t), \quad (3)$$

where Φ is the standard normal CDF, $r(x)$ is the static risk of the action, $\tau(t)$ is the supervisor’s current tolerance, and h is a (typically small) interaction that lets tolerance drift differently for different actions.

The decomposition separates what is a property of the action from what is a property of the supervisor: r can be shared across supervisors (and is what a static corpus such as R-Judge [50] could inform), while τ is one-dimensional, per supervisor, and drifts. The split is identified only up to a shared constant, which the prior fixes.

Proposition 1 (Unary feedback identifies the boundary; pairwise does not). *Let Φ be strictly increasing. (a) If $\Phi(f(x)) = \Phi(g(x))$ for all x , then $f = g$; in particular the allow region $\{x : f(x) > 0\}$ is identified by unary approval probabilities. (b) The pairwise likelihood $\Pr(x \succ x') = \Phi(f(x) - f(x'))$ is invariant under $f \mapsto f + c$ for every constant c ; hence for every f and every x there is a g with identical pairwise likelihoods for all pairs and the opposite sign at x .*

Part (b) says that preference comparisons, however many, cannot tell whether an action lies on the allow or the deny side: only its rank relative to other actions is observable. Unary approve/deny feedback carries exactly the absolute information that pairwise feedback lacks. This corrects the first version of this paper, which described the unary model as “a degenerate pairwise comparison against a fixed reference point”; the reference point is precisely what pairwise data cannot supply.

4 Gaussian-Process Prior and Posterior

We place a GP prior $f \sim \mathcal{GP}(0, k)$ [42]. With feature maps for the action and context, let k_{tool} and k_{ctx} be squared-exponential kernels, $k_x = k_{\text{tool}} k_{\text{ctx}}$, and $k_{\text{time}}(t, t') = \exp(-|t - t'|/\lambda)$. We compare two kernels:

$$k_{\text{prod}}((x, t), (x', t')) = \sigma^2 k_x(x, x') k_{\text{time}}(t, t'), \quad (4)$$

$$k_{\text{add}}((x, t), (x', t')) = \sigma_s^2 k_x(x, x') + \sigma_g^2 k_{\text{time}}(t, t') + \sigma_d^2 k_x(x, x') k_{\text{time}}(t, t'). \quad (5)$$

The product kernel (4) was the kernel of the first version. The additive kernel (5) mirrors Definition 1: the first term is a prior on the static risk r , the second on the shared tolerance τ (every label, for any action, updates it), and the third on the interaction h . Both are positive semidefinite, since sums and Schur products of PSD kernels are PSD.

After observing $\mathcal{D}_N = \{(x_i, t_i, y_i)\}_{i=1}^N$ the posterior $p(f \mid \mathcal{D}_N) \propto \mathcal{GP}(0, k) \prod_i \Phi(f_i)^{y_i} (1 - \Phi(f_i))^{1-y_i}$ is intractable; we use the Laplace approximation (42, Algorithms 3.1–3.2), whose predictive latent mean at x_* is $\mu_* = \mathbf{k}_*^\top \nabla \log p(\mathbf{y} \mid \hat{\mathbf{f}})$ and variance $\sigma_*^2 = k_{**} - \mathbf{k}_*^\top (K + W^{-1})^{-1} \mathbf{k}_*$. Under this Gaussian approximation the predictive approval probability is exactly

$$\hat{p}(x_*) := \mathbb{E}[\Phi(f_*)] = \Phi(\mu_*/\sqrt{1 + \sigma_*^2}). \quad (6)$$

The Laplace approximation is known to underestimate the magnitude of the posterior mean and so to give over-cautious probabilities [31]; we show in Section 11 that the kernel, not the approximation, dominates the calibration error here. Hyperparameters can be chosen by maximizing the Laplace marginal likelihood; Section 11 shows why this average-case criterion should not alone choose the forgetting rate.

5 The Gateway Decision Rule

Let an erroneous ALLOW cost c_{FA} , an erroneous BLOCK cost c_{FB} , and one escalation cost c_{ask} , all positive. For an approval probability p the expected losses are $L_{\text{ALLOW}}(p) = (1 - p) c_{\text{FA}}$, $L_{\text{BLOCK}}(p) = p c_{\text{FB}}$ and $L_{\text{ASK}} = c_{\text{ask}}$.

Proposition 2 (Bayes-optimal three-tier rule). *Let $\tau_{\text{high}} = 1 - c_{\text{ask}}/c_{\text{FA}}$ and $\tau_{\text{low}} = c_{\text{ask}}/c_{\text{FB}}$. (a) For any posterior μ over the latent value and integrable link, the posterior expected loss of each action equals its loss at the posterior-mean approval probability $\hat{p} = \int \Phi d\mu$; hence the Bayes action depends on the posterior only through $\hat{p}$. (b) The band is non-empty, $\tau_{\text{low}} < \tau_{\text{high}}$, iff $c_{\text{ask}}(c_{\text{FA}} + c_{\text{FB}}) < c_{\text{FA}} c_{\text{FB}}$. (c) If it is, the rule*

$$d(\hat{p}) = \begin{cases} \text{ALLOW} & \text{if } \hat{p} > \tau_{\text{high}}, \\ \text{BLOCK} & \text{if } \hat{p} < \tau_{\text{low}}, \\ \text{ASK} & \text{otherwise} \end{cases} \quad (7)$$

attains the minimum expected loss over the three actions for every $\hat{p}$, strictly so off the two thresholds. (d) If it is not, some auto-decision is always at least as good as asking.

The rule of the first version, with band (0.35, 0.65), is exactly this rule with symmetric costs $c_{\text{FA}} = c_{\text{FB}} = c_{\text{ask}}/0.35$. Asymmetric costs give asymmetric bands: with $c_{\text{FA}} = 10$, $c_{\text{FB}} = 4$, $c_{\text{ask}} = 1$ the band is (0.25, 0.90). The thresholds are therefore *specified* by the operator, not tuned on data. This matters because tuning them needs labels on auto-decided actions, which the gateway never obtains (Section 8); the first version tuned its test-phase thresholds with the oracle’s labels on every validation action, information no deployment has.

Proposition 3 (Escalation floor). *Let $q(x) = \Phi(f(x))$ be the true approval probability. Any policy that is pointwise Bayes-optimal given q (that is, with a perfect posterior) must escalate every action with $\tau_{\text{low}} < q(x) < \tau_{\text{high}}$. Consequently, for any finite population of actions with nonnegative weights, its escalated mass is at least the mass of the open band.*

The first version asserted that an 85–90% auto-approval rate “emerges naturally as the posterior concentrates”. Proposition 3 replaces that assertion: the achievable automation rate is bounded by the supervisor’s own ambiguity at the operator’s costs, which no learner can remove. In our simulation the floor is 10.3% at the symmetric band but 31.3% at the safety-weighted band.

Remark 1 (Escalation versus acquisition). *Proposition 2 is a one-step rule: it values the human’s answer only for the action at hand. An answer also improves future decisions, so a fully sequential rule would ask when $c_{\text{ask}} - \text{VOI}(x) < \min(L_{\text{ALLOW}}, L_{\text{BLOCK}})$, with VOI the expected reduction in future loss. The ASK band is therefore an escalation rule first; whether it is also a good acquisition rule is an empirical question, and Section 11 shows that the answer depends on the kernel.*

Remark 2 (Epistemic margin). *Because $\hat{p} = \Phi(\mu/\sqrt{1 + \sigma^2})$, large posterior variance already pulls $\hat{p}$ toward $\frac{1}{2}$. An operator who wants an explicit margin can instead allow only when $\Pr(\Phi(f_*) > \tau_{\text{high}}) \geq 1 - \delta$, which for a Gaussian posterior and strictly increasing Φ is $\mu_* - z_{1-\delta} \sigma_* > \Phi^{-1}(\tau_{\text{high}})$, a lower confidence bound in the style of safe exploration [49].*

6 Non-Stationarity and Forgetting

The time kernel k_{time} is the Ornstein–Uhlenbeck covariance: for fixed x , $f(x, \cdot)$ is a mean-reverting Markov process. It lets recent labels dominate, which tracks drift, but it also reverts every unlabelled action to the prior mean.

Proposition 4 (Forgetting). *Let $\mu(x_*, t) = \sum_i k((x_*, t), (x_i, t_i)) \alpha_i$ be any kernel predictor whose coefficients α are fixed by the training data (the Laplace GP-probit mean has $\alpha = \nabla \log p(\mathbf{y} \mid \hat{\mathbf{f}})$), and let $t \geq \max_i t_i$ and $\Delta \geq 0$. (a) Under k_{prod} , $\mu(x_*, t + \Delta) = e^{-\Delta/\lambda} \mu(x_*, t)$, and the variance reduction $\mathbf{k}_*^\top A \mathbf{k}_*$ (for any fixed A) decays by $e^{-2\Delta/\lambda}$. (b) Hence, if no further labels arrive, Φ is continuous at 0 with $\Phi(0) = \frac{1}{2}$ and $\tau_{\text{low}} < \frac{1}{2} < \tau_{\text{high}}$, every action is eventually escalated; explicitly, an action with $\mu(x_*, t) = m > 0$ is no longer auto-allowed after at most $\lambda \ln(m/z)$ label-free steps, where $z = \Phi^{-1}(\tau_{\text{high}}) > 0$. (c) Under k_{add} , only the time-coupled components decay: $\mu(x_*, t + \Delta) \rightarrow \sum_i \sigma_s^2 k_x(x_*, x_i) \alpha_i$, the static part.*

Part (b) contradicts a claim of the first version, that k_{time} “down-weights stale evidence but does not itself generate new probes”. It does: every region of the action space that stops receiving labels, directly or through its kernel neighbours, drifts back into the band on a fixed clock, whether or not the supervisor’s tolerance has changed. With $\lambda = 90$ and $\tau_{\text{high}} = 0.65$ ($z \approx 0.385$), an action with posterior mean $m = 2$ returns within $90 \ln(2/0.385) \approx 148$ steps. This forced re-escalation is an implicit re-audit, which is useful under drift, but it charges the supervisor for the static structure the model has already learned and it keeps the posterior close to the prior, which makes the gateway underconfident. The additive kernel keeps the static part and forgets only the tolerance and the interaction; the price is that it loses the implicit re-audit, which Section 8 restores explicitly. Note also that the Ornstein–Uhlenbeck process is non-directional: it models change, not the one-way accumulation of trust. A sliding window of recent observations or online changepoint detection [1] are alternatives rather than equivalents.

7 Correlated Generalization

A contextual learner that treats each (a, c) cell independently needs labels in every cell; the kernel transfers evidence between similar actions. With the featurization used here (reversibility, base sensitivity, blast radius and category of the tool; target sensitivity, destructive-argument flag and task of the context), approving `write_file` to `workspace_src` transfers to `write_file` to `workspace_tests`. Transfer cuts both ways: under the isotropic lengthscales of Section 11 the prior correlation of `git_push` and `git_force_push` is 0.975 and that of `write_file` and `delete_file` on the same target 0.87, while moving target sensitivity from a sandbox to production infrastructure changes the context kernel only by a factor 0.71. Approval therefore also propagates toward dangerous neighbours. The claim of the first version that “a new tool with no interaction history inherits the prior μ_0 , which maps to ASK” holds only for actions far from all labelled ones under the kernel; a new action that resembles approved actions inherits their approval. Per-feature lengthscales, a prior that makes f decrease in the risk attributes [44], or the epistemic margin above are the natural remedies; we measure the leak rather than fix it here.

8 Selective Labels and Audits

Only escalated actions are labelled. The GP posterior itself is not biased by this adaptive design when the model is correctly specified, since the decision to ask depends only on past observations; but every frequency computed from the labels is, including false-allow rates and any threshold tuning, and a confidently wrong ALLOW region is never revisited except through forgetting. We therefore add random audits: each auto-decided action is also shown to the human with probability ε .

Proposition 5 (Audits). *(a) Let A_i be audit indicators with $\mathbb{E}[A_i] = \varepsilon > 0$ (independence is not required) and e_i the false-allow indicators of N auto-allowed actions. The Horvitz–Thompson estimator $\widehat{\text{FA}} = \sum_i A_i e_i / (N\varepsilon)$ satisfies $\mathbb{E}\widehat{\text{FA}} = \frac{1}{N} \sum_i e_i$. (b) If the true false-allow rate is at least $\alpha \in (0, 1]$, the probability that n independent audits are all clean is at most $(1 - \alpha)^n \leq e^{-\alpha n}$, which is at most δ once $n \geq \ln(1/\delta)/\alpha$.*

Part (b) is the familiar “rule of three” made exact: certifying a false-allow rate below 1% at 95% confidence takes 300 clean audits. Published evaluations of deployed agent-action classifiers rest on samples of this order or smaller; for example, the false-negative rate reported for one production classifier on real dangerous actions is estimated from 52 examples [28].

9 Relation to Preferential Bayesian Optimization

Table 1 places the gateway next to preferential Bayesian optimization (PBO). The two share a latent function observed through a binary-response likelihood and the same approximate inference. They differ in three structural ways. First, in PBO the learner chooses the query; here the agent’s stream chooses x_t and the learner only chooses whether to ask (stream-based selective sampling). Second, PBO seeks an optimizer, while the gateway seeks a level set, $\{x : f(x) > \Phi^{-1}(\tau)\}$. Third, and most fundamentally, pairwise data identify f only up to a constant (Proposition 1), which suffices to find a maximizer but not a threshold. We therefore regard the gateway as GP classification with a reject option and a level-set objective, a relative of PBO rather than an instance of it. (In González et al. [24] the GP is placed on the duel space $\mathcal{X} \times \mathcal{X}$ with a logistic link; the difference form $\Phi(f(x_i) - f(x_j))$ is that of Chu and Ghahramani [13].)

Component	PBO	Trust calibration (this paper)
Input space	design parameters	(action, context, time)
Who picks the query	learner	agent; learner only decides to ask
Feedback	pairwise preference	unary approve/deny
Identified	f up to a constant	f (Prop. 1)
Objective	$\arg \max f$	level set $\{\hat{p} > \tau_{\text{high}}\}$ etc.
Decision rule	acquisition function	Chow rule (Prop. 2)
Unlabelled decisions	none	ALLOW/BLOCK (selective labels)

Table 1: Structural comparison with preferential Bayesian optimization.

10 Datasets and Evaluation

Empirically grounding the gateway requires proposed agent actions paired with human approve/deny judgments. R-Judge [50] supplies agent interaction records with human safe/unsafe labels and is a natural source for the static risk $r(x)$ of Definition 1. Agent-SafetyBench [51] and ToolEmu [45] widen coverage, but their labels come from automated judges rather than per-action human approval. No public dataset captures the longitudinal, per-supervisor signal the tolerance $\tau(t)$ requires, so we evaluate in a controlled simulation with a known oracle, the standard protocol for GP preference and level-set methods: recovery of the oracle means that the inference is correct under the model, not that the world behaves like the model.

11 Simulation Study

Protocol. The action space has 18 tools with decision-time risk attributes, 8 target-sensitivity tiers and 7 task contexts. The oracle instantiates Definition 1: a static acceptability term linear in the risk attributes, a saturating trust term with an abrupt reset at $t = 750$, a task offset, and a three-way veto (irreversible *and* sensitive *and* low trust) that is active for roughly 90 steps after the reset; approvals are $y \sim \text{Bernoulli}(\Phi(f^*))$. The kernels see the risk attributes but never the trust state or the veto. Each run processes $N = 1500$ decisions prequentially; $[0, 560)$ is warm-up and the rest is scored, split into an early phase containing the reset and a late phase. Thresholds come from costs, never from labels: the symmetric band $(0.35, 0.65)$ and the safety-weighted band $(0.25, 0.90)$. The headline metric is *regret*: the mean Chow loss of the gateway’s decisions minus that of the oracle policy that knows $\Phi(f^*)$, in units of one escalation. We compare the first version’s product kernel ($\sigma^2 = 1.6$, $\lambda = 90$), the additive kernel with a configuration fixed before any version-2 run ($\sigma_s^2 = 1.6$, $\sigma_g^2 = 1.0$, $\sigma_d^2 = 0.6$, $\lambda = 90$), evidence-selected variants of both, a Bayesian linear probit with the same shared drift term (the oracle’s static term is linear in the features by construction), a per-cell Beta-Bernoulli learner (one estimate per tool, target and task) and the first version’s per-tool baseline. Lengthscales are 1.1 (tool) and 1.2 (context) throughout. Tables use 10 seeds; the acquisition probe uses 20 paired seeds. All numbers are generated by the released code.

The additive kernel halves the human burden. Table 2 gives the main comparison. Replacing the product kernel by the additive one cuts regret from 0.137 to 0.056, human labels from 28.4% to 13.5% of actions (a $7.4\times$ rather than $3.5\times$ reduction relative to escalating everything), and the calibration error from 0.213 to 0.104, with a false-allow rate of 1.0% against 1.2%. The price is auto-decision accuracy, which falls from 98.6% to 97.2%, because the additive gateway auto-decides many more contestable actions. In the late phase the additive gateway escalates 11.5% of actions

Model	Regret	ASK %	Auto acc. %	False-allow %	ECE	Veto ALLOW
Per-tool Beta (v1 baseline)	0.660	0.0	67.0	1.7	0.314	15/109
Per-cell Beta	0.330	9.1	81.7	2.0	0.281	14/109
Product, $\lambda=90$ (v1)	0.137	28.4	98.6	1.2	0.213	8/109
Product, evidence-selected	0.063	14.6	97.8	1.0	0.172	13/109
Linear probit + drift, ev.-sel.	0.059	10.0	96.7	2.0	0.062	31/109
Additive, $\lambda=90$ (v2)	0.056	13.5	97.2	1.0	0.104	11/109
Additive, evidence-selected	0.059	11.9	96.8	1.5	0.093	20/109

Table 2: Symmetric costs, no audits, scored phases, mean over 10 seeds. Regret is the cost-weighted headline; ASK, accuracy and false-allow (share of ALLOWS the oracle would deny) are its components. The last column pools, over seeds, the ALLOWS among the few actions for which the veto is active; read it as directional. The perfect-posterior escalation floor is 10.3%.

Model	ASK %	Auto acc. %	Regret
Product, $\lambda=90$ (v1)	99.1	99.2	0.509
Product, evidence-selected	55.3	99.9	0.171
Linear probit + drift, ev.-sel.	33.1	99.4	0.071
Additive, $\lambda=90$ (v2)	45.9	99.7	0.113
Additive, evidence-selected	41.5	99.6	0.092

Table 3: Safety-weighted costs ($c_{FA} = 10$, $c_{FB} = 4$, $c_{ask} = 1$; band (0.25, 0.90)), 10 seeds. Floor: 31.3%.

against a floor of 10.3%; the product gateway escalates 25.2%. Because the oracle is itself additive, the size of this advantage is indicative; what should transfer is the structural point that only the drifting components of a model should be forgotten. The simple baselines show why a structured model is needed at all: the per-tool learner auto-decides everything at 67.0% accuracy (regret 0.660), and the per-cell learner, which needs a label in every cell, reaches 81.7% (regret 0.330).

The linear baseline and the interaction. The Bayesian linear probit with the same shared drift is competitive on average metrics (regret 0.059, ECE 0.062), which is expected because the oracle’s static term is linear in the features by construction. It cannot represent the veto interaction, and it allows 31/109 veto-window actions against 11/109 for the additive kernel. We read this as evidence that the kernel’s structure (what is forgotten) matters more than its flexibility for average burden, while the non-parametric interaction term earns its keep on the rare, safety-relevant region; on real data the linear static term is unlikely to be correct.

Evidence selection optimizes average fit, not tail safety. On the labels the first-version gateway collected, the Laplace marginal likelihood ranks the first-version kernel last (mean log evidence -276.1 against -247.1 for the additive kernel, with the linear probit best in 8 of 10 seeds; this is evidence under a selected design, since only escalated actions are labelled). Selecting hyperparameters by evidence during the run chooses long memories (typically $\lambda = 480$ for the additive and 480–960 for the product kernel) and improves calibration, but it nearly doubles veto-window allows for the additive kernel (20/109 against 11/109). The marginal likelihood is an average-case criterion and the veto window holds about one percent of scored actions: it is a good criterion for structure and a poor one, on its own, for memory length. We therefore keep the pre-specified $\lambda = 90$.

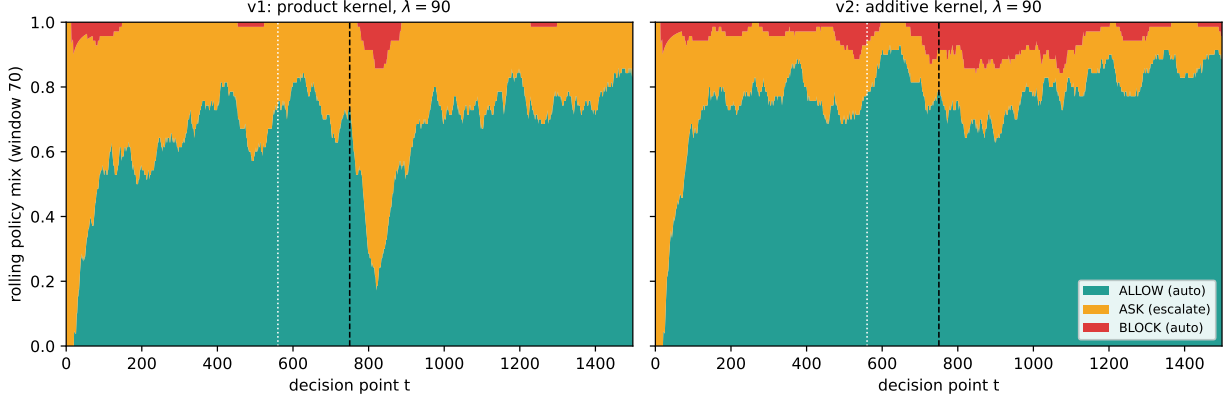


Figure 1: Rolling policy mix for one seed. Left: the first version’s product kernel; right: the additive kernel. The dashed line is the trust reset at $t = 750$, the dotted line the end of the warm-up.

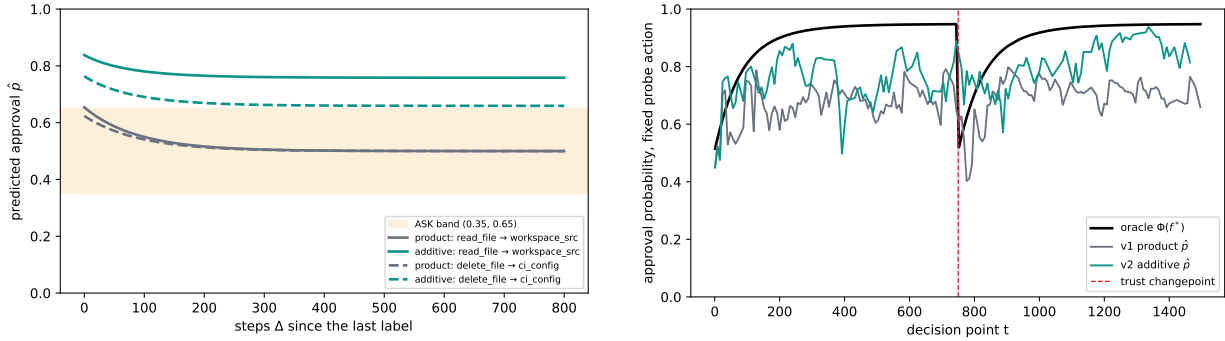


Figure 2: Left: Proposition 4 on a fixed training set; the product kernel returns every action to $\hat{p} = \frac{1}{2}$, inside the band, while the additive kernel keeps the static part. Right: a fixed probe action under drift and the reset; both kernels track the reset, the additive one from a less underconfident level.

Forgetting makes a high-confidence bar unreachable. Under safety-weighted costs (Table 3) the product gateway escalates 99.1% of all actions (regret 0.509): its posterior never becomes confident enough to clear $\hat{p} > 0.9$, because every action it stops asking about decays toward $\hat{p} = \frac{1}{2}$ (Proposition 4). The additive gateway escalates 45.9% (regret 0.113) against a floor of 31.3%. Figure 2 (left) isolates the mechanism: with the training set fixed, the product kernel’s prediction for a benign `read_file` decays from 0.65 to 0.50 as the time since the last label grows, while the additive kernel’s settles at its static part, 0.76.

Why uncertainty-targeted querying failed, and why it now works. The first version reported that querying inside the ASK band was no more sample-efficient than random querying at a matched budget, attributed this to class imbalance, and proposed an information-theoretic acquisition rule as the remedy. Table 4 tests both claims with 20 paired seeds. With the product kernel the band is indeed no better than random (-0.5 and $+0.2$ percentage points of prequential accuracy on contestable actions, standard errors 0.7 and 0.6), on both a fully stationary oracle and the reset oracle, so non-stationarity was never the cause. With the additive kernel the same

Kernel	Oracle	Band	Random	BALD	Band-random	BALD-random
Product, $\lambda=90$ (v1)	stationary	82.0	82.4	81.2	-0.5 ± 0.7	-1.2 ± 0.8
Product, $\lambda=90$ (v1)	changepoint	77.4	77.3	75.9	$+0.2 \pm 0.6$	-1.4 ± 1.0
Additive, $\lambda=90$ (v2)	stationary	84.2	80.7	79.1	$+3.5 \pm 1.7$	-1.6 ± 2.0
Additive, $\lambda=90$ (v2)	changepoint	77.6	74.5	76.0	$+3.1 \pm 1.2$	$+1.5 \pm 1.3$

Table 4: Acquisition probe: prequential accuracy (%) on contestable actions ($\Phi(f^*) \in [0.15, 0.85]$) at a matched label budget; differences in percentage points $\pm$ paired standard error over 20 seeds.

Model	Labels/action %	Regret (audit billed)	Veto ALLOW	HT est./truth %	CI covers
Product, $\lambda=90$ (v1)	28.4 $\rightarrow$ 30.2	0.137 $\rightarrow$ 0.119 (0.157)	8/109 $\rightarrow$ 7/109	5.9 / 6.1	100%
Linear probit + drift, ev.-sel.	10.0 $\rightarrow$ 15.7	0.059 $\rightarrow$ 0.043 (0.088)	31/109 $\rightarrow$ 14/109	6.3 / 6.8	100%
Additive, $\lambda=90$ (v2)	13.5 $\rightarrow$ 18.2	0.056 $\rightarrow$ 0.046 (0.089)	11/109 $\rightarrow$ 5/109	7.2 / 6.4	100%
Additive, evidence-selected	11.9 $\rightarrow$ 16.2	0.059 $\rightarrow$ 0.054 (0.098)	20/109 $\rightarrow$ 20/109	10.0 / 7.4	90%

Table 5: Random audits of $\varepsilon = 5\%$ of auto-decided actions, symmetric costs, 10 seeds; arrows show no audits $\rightarrow$ audits. Regret charges decisions only, so audits are free in it; the value in parentheses bills each audit as one escalation. The last columns compare the Horvitz–Thompson estimate of the realized false-allow rate with its true value, and give the share of seeds whose Jeffreys 95% interval on the audited fraction covers the truth.

band beats random by $+3.5$ (± 1.7) and $+3.1$ (± 1.2) points. A stream-based BALD sampler [27], which labels actions whose epistemic information is in the top fraction of a recent window, does not improve on the band under either kernel. We therefore retract both parts of the first version’s explanation: the controlling variable is forgetting, and the fix is the kernel, not the acquisition rule. We also retract a claim made in our own review of the first version, that the band fails because it spends labels on intrinsically ambiguous actions: under the product kernel the band does place 27.1% of its labels on actions with true approval probability in $[0.35, 0.65]$ (base rate 10.4%), at the same mean posterior variance as random querying (1.32 against 1.33), but under the additive kernel it places even more of its labels there and still beats random.

Audits. Table 5 adds random audits at $\varepsilon = 5\%$. For the additive gateway they raise labels per action from 13.5% to 18.2% and halve veto-window allows (11/109 $\rightarrow$ 5/109); for the linear baseline they cut veto-window allows from 31/109 to 14/109. Regret as defined charges decisions only, so an audit of an auto-decided action is free in it; on that measure the additive gateway’s regret falls from 0.056 to 0.046. Billing each audit as one escalation, it rises to 0.089 instead: audits pay for themselves in average regret only if one costs at most 0.22 of an escalation, which a post-hoc review may or may not. Audits are justified by what they make estimable and by the veto-window safety, not by average regret. The Horvitz–Thompson estimate of the realized false-allow rate is 7.2% against a true 6.4%, and a Jeffreys interval on the audited fraction covers the truth in 100% of seeds. With about forty audits per seed the estimate is noisy, as Proposition 5(b) predicts: audits make the error rate estimable, not small.

Correlated generalization cuts both ways. We hold out two combinations and never label them. On the benign `write_file` $\rightarrow$ `workspace_tests` the additive gateway auto-decides 96% of occurrences correctly by kernel transfer (product 81%, per-cell 0%, since it must ask about every unseen cell). On the dangerous `git_force_push` $\rightarrow$ `prod_infra`, where the oracle denies 64 of the occurrences, every kernel model wrongly allows some: 7 (additive), 8 (product) and 8 (linear),

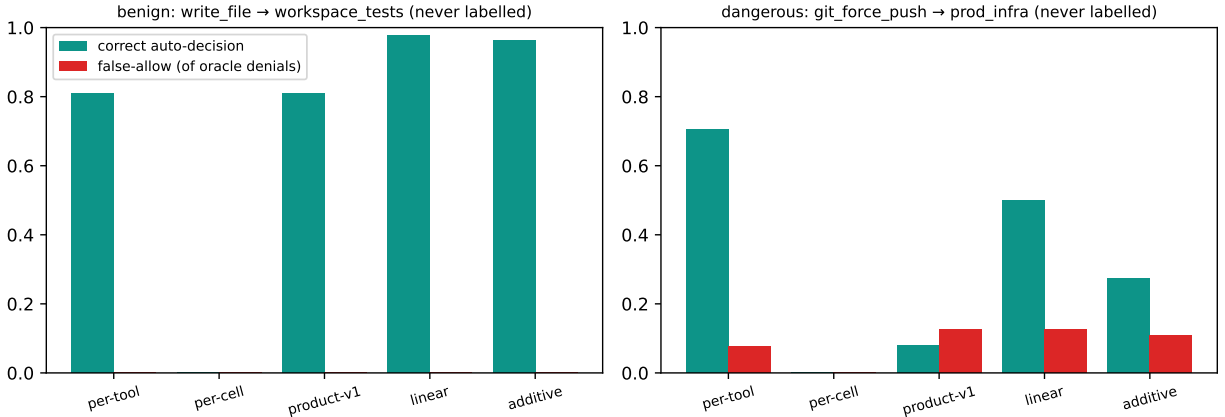


Figure 3: Held-out combinations that are never labelled; prequential decisions over 10 seeds. Left: benign transfer. Right: a dangerous combination; red bars are the share of the oracle’s denials that were auto-allowed.

against 5 for the per-tool learner, which has labels on the same tool at other targets, and none for the per-cell learner, which asks. Figure 3 shows both directions. This is the leak Section 7 predicts; we report it rather than tune it away.

12 Beyond a Known Tool Set: Calibrating an Opaque Judge

The gateway above assumes a fixed taxonomy of tools whose risk attributes are known. Current agent harnesses increasingly route work through a general-purpose shell, so an action is arbitrary code and no fixed taxonomy exists. In practice the ASK decision is then delegated to a second model, an LLM *judge*. In Claude Code’s auto mode, for example, a transcript classifier reviews shell commands, web fetches, external tool calls and filesystem operations outside the project (read-only tools and in-project edits skip it); it “sees only user messages and the agent’s tool calls”, with the agent’s own messages and tool outputs removed; it returns allow or block, a block comes back to the agent as a tool result telling it to find a safer path, and the human is asked only after three consecutive or twenty total blocks [28]. Its reported false-positive rate is 0.4% on 10,000 real actions and its false-negative rate 17% on 52 real overeager actions and 5.7% on 1,000 synthetic exfiltration attempts [28]. Two questions follow: is the judge trustworthy, and when it says allow or block, how likely is that verdict to be what the supervisor would have decided? The judge is a black box; only its verdict (or score) and, on some actions, the supervisor’s decision are observable.

OpenAI’s open-source Codex CLI makes different choices [39]. Its auto-review reviewer is a sub-agent consulted only when the static layers (sandbox, approval policy, command rules) would otherwise ask the user. It sees user and developer messages, project instruction files, the agent’s messages, tool calls and tool outputs, but not the agent’s reasoning; everything other than user and developer messages is marked as untrusted evidence, and the reviewer may run read-only commands to investigate. It returns allow or deny with an optional risk level, and its experimental asynchronous pre-scorer maps a single output token to a score of 0 or 1. A denial returns to the agent as a rejection, and three consecutive denials, or ten among the last fifty reviews, interrupt the turn. The two designs sit at opposite ends of a trade-off in the judge’s view: a reasoning-blind view limits what injected text can reach the judge, while a fuller view exposes more of the context that separates safe from unsafe actions, and more untrusted text with it. Neither exposes a calibrated

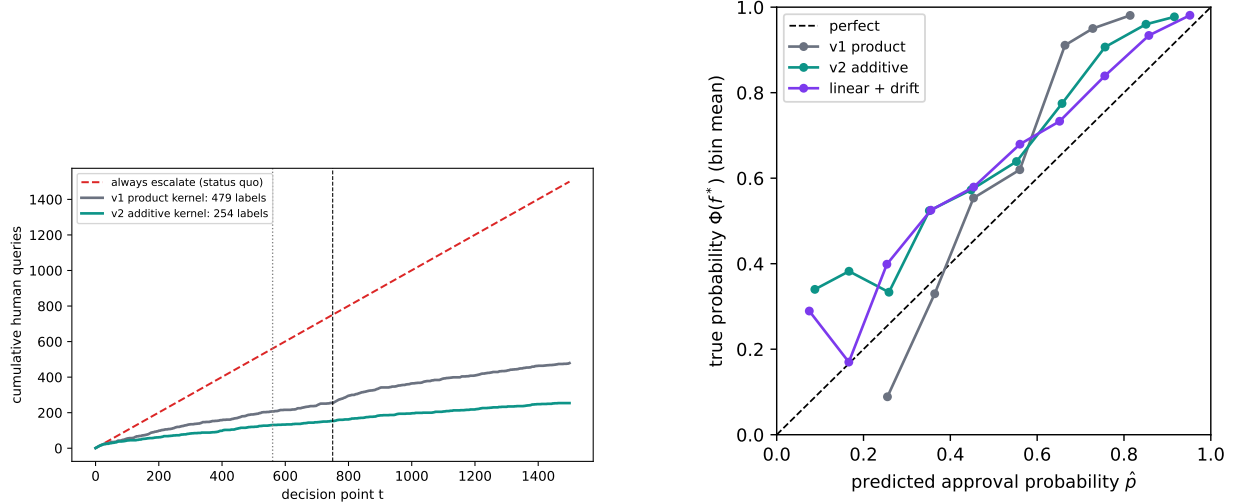


Figure 4: Left: cumulative human labels for one seed against escalating every action. Right: reliability of $\hat{p}$ against the true approval probability (one seed, scored phases).

score, and neither routes judge-allowed actions to a human, which is why the verdict-only condition of our simulation is the realistic one and why audits are needed below.

Definition 2 (Judge reliability for a supervisor). *For a judge J with verdicts in $\{ALLOW, BLOCK\}$ and a supervisor with approval probability $q(x, t)$, the judge’s false-allow and false-block rates over a period are $FA_J = \Pr(y = 0 \mid J = ALLOW)$ and $FB_J = \Pr(y = 1 \mid J = BLOCK)$. The judge is (α, δ) -trustworthy for that supervisor and period if $FA_J \leq \alpha$ is certified with confidence $1 - \delta$. The reliability of an individual verdict is the posterior $\hat{p}(J(x), z(x), t) = \Pr(y = 1 \mid J(x), z(x), t, \mathcal{D})$ given the verdict, cheap observable features z and the labels so far.*

Proposition 6 (What cannot be learned without paired labels). *(a) For every judge allow rate $a \in (0, 1)$ and every $\varphi \in [0, 1]$ there is a joint distribution of verdict and supervisor decision with $\Pr(J = ALLOW) = a$ and $FA_J = \varphi$. (b) Even the two marginals together do not identify FA_J : two joints with the same judge allow rate (0.8) and the same supervisor approval rate (0.9) have $FA_J = 0$ and $FA_J = 1/8$.*

A judge’s error rate for a supervisor is therefore a property of the *joint* distribution, observable only through actions that carry both a verdict and a human label. Several judges do not remove the need: latent-class models become generically identifiable from three or more observers only if they are conditionally independent given the truth [2, 17], and the errors of language models are strongly correlated [23, 30].

Proposition 7 (Reliability is relational). *Fix a judge, i.e. the set A of actions it allows, and a finite population with nonnegative weights. If supervisor q' is nowhere more permissive than supervisor q on A , the judge’s false-allow rate for q' is at least its false-allow rate for q .*

The same, unchanged judge is thus more or less trustworthy depending on whose decisions it stands in for and when: a certificate for one supervisor or one period does not transfer to another. This is where the judge and the gateway meet. The judge can estimate the static risk $r(x)$ of arbitrary code, which is what it is good at; the published analysis of the production classifier finds that in most misses it “correctly identifies the danger” but misjudges whether “a real consent

signal in the session actually covers this action” [28], which is the supervisor-specific tolerance $\tau(t)$ of Definition 1.

Proposition 8 (Unbiased estimation with known propensities). *If action i receives a human label with known probability $\pi_i > 0$ (for example $\pi_i = 1$ when it is escalated and $\pi_i = \varepsilon$ when it is an audited auto-decision), the Horvitz–Thompson sum $\sum_i L_i e_i / \pi_i$ is unbiased for $\sum_i e_i$.*

Proposition 8 generalizes Proposition 5(a) and lets escalations and audits both count, provided each label’s propensity is logged; this is the logic of active and prediction-powered inference [4, 52]. In a design where the human sees an action only after repeated judge blocks, labels exist only for blocked actions, every propensity on judge-allowed actions is zero, and FA_J is not estimable at all.

A calibrated gateway on top of the judge. We therefore treat the judge’s output as a feature rather than a decision. The gateway of Sections 4–8 runs on $(J(x), t)$ with the additive kernel: its static part learns how this supervisor’s approvals relate to the judge’s output in each category, and its time part tracks the supervisor’s tolerance. Its posterior $\hat{p}$ answers the second question per action; its Chow rule escalates rather than blocks where the judge’s verdict is unreliable for this supervisor; and audits with logged propensities keep FA_J estimable. One limit is structural: the gateway cannot separate actions that look identical in its own features, so it cannot repair a judge blind spot its features do not expose. Argument-level features (for example static patterns in the command, as in the Go port’s shell heuristics) are the remedy, not more calibration. The released Go port implements this gateway for real harnesses: a featurizer over the judge’s verdict and a command category, cost-derived thresholds, and audits whose sampling propensities are logged with each label.

Simulation. We keep the oracle, stream and seeds of Section 11 and add a judge, with every setting fixed in a pre-registration committed before the first run (no deviations). The judge sees the action’s static risk but not the supervisor’s trust state, task preferences or veto; it blocks when a noisy score of the risk exceeds a fixed threshold, and it has a blind spot: destructive shell commands look benign to it. The gateway sees only the judge’s verdict (or score), an eight-way command category and time. This simulated judge is much stricter than the production classifier: it blocks about a quarter of all actions, most of which this supervisor would approve, against a reported false-positive rate of 0.4%. Gains from turning its blocks into escalations are therefore dominated by avoided false blocks and reflect the pre-registered threshold rather than a prediction about any deployed system; the relational and estimation results below do not depend on it.

Results. Table 6 summarizes the policies. Letting the judge decide alone gives regret 0.296 with no human labels; escalating its blocks instead of enforcing them gives 0.088 at 25.3% labels. The calibrated gateway reaches the same regret (0.086) with half the labels (11.8%), and lower regret with the judge’s score (0.068) or with audits (0.075, audits unbilled). It is also more permissive than the judge (false-allow 2.2% against 0.9%), because it learns that this supervisor approves much of what the judge blocks: the gateway estimates whether *this supervisor* would approve, not whether the action is safe, and a judge’s fixed policy can be safer than a supervisor’s revealed tolerance. Under safety-weighted costs the sign flips (false-allow 0.1%) at 46.7% labels, and calibration buys regret 0.150 against 0.173 for escalating the judge’s blocks, at about twenty more labels per hundred actions. With the product kernel the gateway again escalates almost everything under safety costs (93.7% labels), the forgetting collapse of Proposition 4 outside the tool taxonomy.

Policy	Symmetric costs				Safety costs	
	Regret	Labels %	FA %	Blind FA	Regret	Labels %
Judge only (allow/block)	0.296	0.0	0.9	28/29	0.565	0.0
Judge, escalate instead of block	0.088	25.3	0.9	28/29	0.173	25.3
Always ask	0.720	100.0	–	0/29	0.516	100.0
Calibrated gateway	0.086	11.8	2.2	23/29	0.150	46.7
Calibrated gateway + 5% audits	0.075	16.5	2.3	26/29	0.168	52.4
Calibrated (product kernel) + audits	0.146	27.2	2.6	17/29	0.462	93.7

Table 6: Opaque-judge study, verdict input, 10 seeds. FA is the share of `ALLOW`s the oracle would deny; Blind FA pools, over seeds, the blind-spot actions the oracle would deny that were auto-allowed. Audits are free in regret (Section 11).

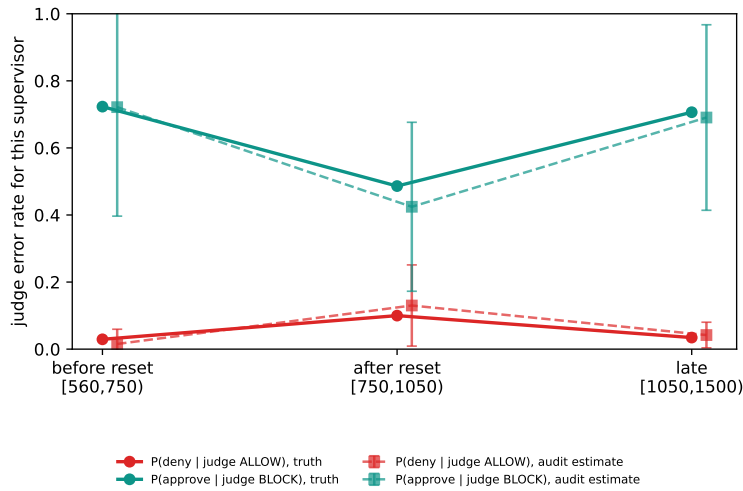


Figure 5: The same judge’s error rates for one supervisor, before and after the supervisor’s trust reset: truth (solid) and Horvitz–Thompson audit estimates (dashed, ± 1 s.d. across seeds).

The judge’s reliability for this supervisor moves while the judge does not (Figure 5): its false-allow rate is 2.9% before the supervisor’s trust reset, 10.0% after it and 3.4% late, as Proposition 7 predicts. Horvitz–Thompson estimates from the gateway’s labels track these values (1.5%, 13.0% and 4.2% on average), unbiased but noisy within a window, with standard deviations across seeds of 4.4 to 12.1 points, because each window holds only 8 to 24 human labels on judge-allowed actions, escalations and audits together. Without audits they are undefined in every seed. The blind spot is not caught under symmetric costs: 23/29 of the blind-spot actions this supervisor would deny are still allowed, because they are indistinguishable from benign shell commands in the gateway’s features; under safety costs the count falls to 3/29 only because the gateway escalates judge-allowed shell commands as a class.

Answering the two questions. Whether a judge is trustworthy is a statistical claim about a moving target. It cannot be read off the judge’s verdicts (Proposition 6); it changes with the supervisor (Proposition 7); and certifying it takes clean audits in proportion to $1/\alpha$ (Proposition 5(b)): 12 for $\alpha = 25\%$, 300 for $\alpha = 1\%$, while the quantity drifts on the supervisor’s timescale. Certification and tracking pull against each other, and the honest form of a trust claim is “false-allow

rate below α for this supervisor over the last n audits”. How likely a given verdict is to be right is the calibrated posterior $\hat{p}$, which needs the same labels and cannot exceed what the features can distinguish.

Limits of the human as ground truth. Our supervisor is noisy but unbiased. Real supervisors are not: users approve 97% of permission prompts in one deployment, and in a controlled study human reviewers caught 13.6% of dangerous commands against 89% for the classifier, with their block rate falling from about 17% early in a session to about 5% after fifty or more prompts [41], consistent with habituation to repeated warnings [3]. A gateway calibrated to a rubber-stamping supervisor learns to rubber-stamp; a fatigue-aware model would make the cost of asking, and the noise of the answer, grow with the escalation rate, which argues for fewer and better-targeted escalations. Judges are also attack surfaces: agent-written prompt injections obtained arbitrary shell execution past two production classifiers in 79% of trials [43], and reliability estimated from benign audits says nothing about adaptive attacks; control protocols with trusted monitors and audit budgets address that setting [6, 26]. Finally, each judge call is itself a paid model call, so a cascade that consults the judge only when the gateway is uncertain, in the spirit of selective evaluation with escalation [29], is a natural next step.

13 Mechanized Verification

The decision-theoretic and algebraic cores of Propositions 1–8 are formalized in Lean 4 (v4.34.1) with Mathlib (v4.34.1) in the `lean/` directory of the released code; the build has no `sorry`, and every theorem depends only on Lean’s standard axioms (`propext`, `Classical.choice`, `Quot.sound`). Table 7 maps each proposition to its theorems. The formalization keeps the link function abstract (strictly increasing, or continuous at zero with value one half) rather than using the Gaussian CDF, which covers the probit and logistic links alike. It does not formalize the Laplace approximation itself: Proposition 4 is stated for any predictor that is linear in the cross-covariance vector with coefficients fixed by the training data, which is the form of the Laplace mean and of exact GP regression. Proposition 2(a) is proved for an arbitrary probability measure over the latent value using the Bochner integral, and Proposition 3 for finite action populations; Proposition 6 is proved by explicit construction on the four cells of the verdict–decision table. The Python experiment checks the same statements numerically (`experiment/tests`).

14 Concluding Remarks

Deciding when an agent may act alone is not a threshold to hand-tune but a latent tolerance to learn, and the decision on top of it is a reject option whose thresholds follow from what errors and interruptions cost. Three structural facts shape what such a gateway can achieve: unary feedback is what identifies the allow/deny boundary; a model that forgets everything charges the supervisor again for what it already knew and cannot reach a demanding confidence bar; and the labels the gateway collects are selected, so audits are needed to know its error rate. In simulation, a kernel that separates static risk from drifting tolerance halves the human burden and turns the ASK band into an effective acquisition rule, while correlated generalization still leaks approval to unseen dangerous actions. When the tool taxonomy is replaced by an opaque judge over arbitrary shell actions, the same machinery calibrates the judge to a supervisor, but the judge’s trustworthiness becomes a moving, relational quantity that only audited labels can measure, and calibration cannot

Result	File	Theorems
Prop. 1	Identifiability.lean	unary_identifies, unary_identifies_boundary, pairwise_shift_invariant, pairwise_cannot_identify_threshold
Prop. 2	Chow.lean	expectedLoss_allow, expectedLoss_block, band_nonempty_iff, allow/block/ask_strictly_optimal, decide_optimal, ask_never_needed
Prop. 3	Floor.lean	optimal_asks_on_band, escalation_floor
Prop. 4	Forgetting.lean	kT_shift, meanProd_shift, quadForm_shift, exp_decay_tendsto, eventually_ask, reentry_bound, reentry_prob, meanAdd_shift, meanAdd_tendsto
Prop. 5	Audit.lean	audit_unbiased, one_sub_pow_le_exp, certify_sample_size
Props. 6–8	Judge.lean	judge_unidentified, judge_marginals_not_enough, judgeFA_mono, ht_unbiased
Remark (margin)	Identifiability.lean	lcb_iff

Table 7: Propositions and their machine-checked counterparts (31 theorems).

see past the features it is given. The main remaining gap is empirical: a longitudinal study of real supervisors, whose own reliability as ground truth declines as they are asked more often.

A Proofs

Proposition 1. (a) Φ is injective, so $\Phi(f(x)) = \Phi(g(x))$ gives $f(x) = g(x)$ for every x . (b) $(f(x) + c) - (f(x') + c) = f(x) - f(x')$. Given x , take $g = f + c$ with $c = -f(x) - 1$ if $f(x) > 0$ and $c = -f(x) + 1$ otherwise; then g has the same pairwise likelihoods and $g(x) = \mp 1$.

Proposition 2. (a) Each loss is affine in p , so by linearity of the integral $\int L_{\text{ALLOW}}(\Phi) d\mu = (1 - \int \Phi d\mu) c_{\text{FA}}$ and similarly for BLOCK. (b) Clear denominators in $c_{\text{ask}}/c_{\text{FB}} < 1 - c_{\text{ask}}/c_{\text{FA}}$. (c) ALLOW beats ASK iff $p > \tau_{\text{high}}$ and BLOCK beats ASK iff $p < \tau_{\text{low}}$; ALLOW beats BLOCK iff $p > c_{\text{FA}}/(c_{\text{FA}} + c_{\text{FB}})$, and when the band is non-empty this point lies strictly inside it, so the pairwise comparisons compose to rule (7). (d) $\min(L_{\text{ALLOW}}, L_{\text{BLOCK}})$ is at most the convex combination with weights $c_{\text{FB}}/(c_{\text{FA}} + c_{\text{FB}})$ and $c_{\text{FA}}/(c_{\text{FA}} + c_{\text{FB}})$, which equals $c_{\text{FA}}c_{\text{FB}}/(c_{\text{FA}} + c_{\text{FB}}) \leq c_{\text{ask}}$ when the band is empty.

Proposition 3. Inside the open band ASK is strictly better than both auto-decisions (Proposition 2(c)), so a pointwise-optimal policy cannot choose either; the band is therefore a subset of the escalated set, and nonnegative weights give the inequality.

Proposition 4. For $s \leq t$ and $\Delta \geq 0$, $|t + \Delta - s| = \Delta + |t - s|$, so $k_{\text{time}}(t + \Delta, s) = e^{-\Delta/\lambda} k_{\text{time}}(t, s)$. Every term of the product-kernel predictor carries one such factor, giving (a) for the mean, and the quadratic form carries two, giving $e^{-2\Delta/\lambda}$. For (b), $\sqrt{1 + \sigma^2} \geq 1$, so $|\mu/\sqrt{1 + \sigma^2}| \leq e^{-\Delta/\lambda} |m| \rightarrow 0$ and continuity gives $\hat{p} \rightarrow \frac{1}{2}$, which eventually lies in any open band around $\frac{1}{2}$. For the explicit time, $\Delta \geq \lambda \ln(m/z)$ gives $e^{-\Delta/\lambda} m \leq z$, and monotonicity of Φ gives $\hat{p} \leq \Phi(z) = \tau_{\text{high}}$. (c) Only the second and third components of k_{add} carry the time factor.

Proposition 5. (a) By linearity, $\mathbb{E} \sum_i A_i e_i = \varepsilon \sum_i e_i$. (b) If the rate is at least α , each audit is clean with probability at most $1 - \alpha$, and $1 - \alpha \leq e^{-\alpha n}$; then $e^{-\alpha n} \leq \delta$ iff $n \geq \ln(1/\delta)/\alpha$.

B Changes from Version 1

1. Version 1 called the gateway an instance of preferential Bayesian optimization and the unary model a degenerate pairwise comparison. Pairwise feedback cannot identify the threshold (Proposition 1); the gateway is GP classification with a reject option.
2. Version 1 gave no loss for the three-tier rule and tuned its test-phase thresholds with oracle labels on all validation actions. The rule is Chow’s (Proposition 2) and thresholds now follow from stated costs.
3. Version 1 asserted that an 85–90% auto-approval rate emerges without tuning. The achievable rate is bounded by the supervisor’s ambiguity (Proposition 3).
4. Version 1 stated that k_{time} does not generate new probes. It forces periodic re-escalation (Proposition 4), which is the main cause of its high escalation rate and underconfidence; version 1 attributed the underconfidence to the Laplace approximation.
5. Version 1 attributed the acquisition result to class imbalance and proposed an information-theoretic rule as the remedy. The cause is forgetting; with an additive kernel the band beats random querying, and a BALD sampler does not help (Table 4).
6. Version 1 reported “508 vs 940 queries, a $\sim 1.8\times$ reduction”; its numerator counted the whole stream and its denominator only the scored phases. On matching phases the version-1 protocol gives about $3.0\times$; the version-2 protocol gives $3.5\times$ (product) and $7.4\times$ (additive).
7. Version 1’s transfer test held out only a benign combination, and its per-tool baseline ignored target and task. Version 2 adds a dangerous hold-out, a per-cell baseline and a linear-probit baseline.
8. Version 1 did not state its hyperparameters, described a sliding window as equivalent to the time kernel, and illustrated argument-level transfer (DROP to TRUNCATE) that its single destructive-argument feature cannot express. These are corrected.

C Additions in Version 3

Section 12 (an opaque judge in front of the gateway, Definition 2, Propositions 6–8 and a pre-registered simulation) and its Lean formalization (`Judge.lean`) are new in version 3.

References

- [1] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*, 2007. URL <https://arxiv.org/abs/0710.3742>.
- [2] Elizabeth S. Allman, Catherine Matias, and John A. Rhodes. Identifiability of parameters in latent structure models with many observed variables. *The Annals of Statistics*, 37(6A): 3099–3132, 2009. doi: 10.1214/09-AOS689.
- [3] Bonnie Brinton Anderson, C. Brock Kirwan, Jeffrey L. Jenkins, David Eargle, Seth Howard, and Anthony Vance. How polymorphic warnings reduce habituation in the brain: Insights from an fMRI study. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pages 2883–2892, 2015. doi: 10.1145/2702123.2702322.
- [4] Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023. doi: 10.1126/science.adi6000.

- [5] Alessio Benavoli and Dario Azzimonti. A tutorial on learning from preferences and choices with Gaussian processes. *Foundations and Trends in Machine Learning*, 19(1):1–120, 2026. doi: 10.1108/ftmal-03-2025-0124.
- [6] Aryan Bhatt, Cody Rushing, Adam Kaufman, Tyler Tracy, Vasil Georgiev, David Matolcsi, Akbir Khan, and Buck Shlegeris. Ctrl-Z: Controlling AI agents via resampling, 2025.
- [7] Ilija Bogunovic, Jonathan Scarlett, and Volkan Cevher. Time-varying Gaussian process bandit optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 51 of *PMLR*, pages 314–323, 2016. URL <https://proceedings.mlr.press/v51/bogunovic16.html>.
- [8] Ilija Bogunovic, Jonathan Scarlett, Andreas Krause, and Volkan Cevher. Truncated variance reduction: A unified approach to Bayesian optimization and level-set estimation. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/hash/ce78d1da254c0843eb23951ae077ff5f-Abstract.html.
- [9] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv preprint arXiv:1012.2599*, 2010. URL <https://arxiv.org/abs/1012.2599>.
- [10] Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krasheninnikov, Lauro Langosco, Zhonghao He, Yawen Duan, Micah Carroll, Michelle Lin, Alex Mayhew, Katherine Collins, Maryam Molamohammadi, John Burden, Wanru Zhao, Shalaleh Rismani, Konstantinos Voudouris, Umang Bhatt, Adrian Weller, David Krueger, and Tegan Maharaj. Harms from increasingly agentic algorithmic systems. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 651–666. ACM, 2023. doi: 10.1145/3593013.3594033. URL <https://doi.org/10.1145/3593013.3594033>.
- [11] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Blumke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. Visibility into AI agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 958–973. ACM, 2024. doi: 10.1145/3630106.3658948. URL <https://doi.org/10.1145/3630106.3658948>.
- [12] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [13] Wei Chu and Zoubin Ghahramani. Preference learning with Gaussian processes. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 137–144. ACM Press, 2005. doi: 10.1145/1102351.1102369. URL <https://doi.org/10.1145/1102351.1102369>.
- [14] David Cohn, Les Atlas, and Richard Ladner. Improving generalization with active learning. *Machine Learning*, 15(2):201–221, 1994. doi: 10.1007/BF00993277.
- [15] Sanjoy Dasgupta. Two faces of active learning. *Theoretical Computer Science*, 412(19):1767–1781, 2011. doi: 10.1016/j.tcs.2010.12.054.

- [16] Sanjoy Dasgupta and Daniel Hsu. Hierarchical sampling for active learning. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 208–215, 2008. doi: 10.1145/1390156.1390183.
- [17] A. P. Dawid and A. M. Skene. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28(1):20–28, 1979. doi: 10.2307/2346806.
- [18] Ewart J. de Visser, Marieke M. M. Peeters, Malte F. Jung, Spencer Kohn, Tyler H. Shaw, Richard Pak, and Mark A. Neerincx. Towards a theory of longitudinal trust calibration in human–robot teams. *International Journal of Social Robotics*, 12(2):459–478, 2020. doi: 10.1007/s12369-019-00596-x. URL <https://doi.org/10.1007/s12369-019-00596-x>.
- [19] David K. Duvenaud, Hannes Nickisch, and Carl Edward Rasmussen. Additive Gaussian processes. In *Advances in Neural Information Processing Systems*, volume 24, 2011. URL https://proceedings.neurips.cc/paper_files/paper/2011/hash/4c5bde74a8f110656874902f07378009-Abstract.html.
- [20] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010. URL <https://jmlr.org/papers/v11/el-yaniv10a.html>.
- [21] Aditya Gangrade, Anil Kag, Ashok Cutkosky, and Venkatesh Saligrama. Online selective classification with limited feedback. In *Advances in Neural Information Processing Systems*, volume 34, pages 14529–14541, 2021.
- [22] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.
- [23] Shashwat Goel, Joschka Strüder, Ilze Amanda Auzina, Karuna K. Chandra, Ponnurangam Kumaraguru, Douwe Kiela, Ameya Prabhu, Matthias Bethge, and Jonas Geiping. Great models think alike and this undermines AI oversight. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 19621–19678, 2025.
- [24] Javier González, Zhenwen Dai, Andreas Damianou, and Neil D. Lawrence. Preferential Bayesian optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1282–1291. PMLR, 2017. URL <https://proceedings.mlr.press/v70/gonzalez17a.html>.
- [25] Alkis Gotovos, Nathalie Casati, Gregory Hitz, and Andreas Krause. Active learning for level set estimation. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1344–1350, 2013. URL <https://www.ijcai.org/Proceedings/13/Papers/202.pdf>.
- [26] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 16295–16336, 2024.
- [27] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*, 2011. URL <https://arxiv.org/abs/1112.5745>.

- [28] John Hughes. How we built Claude Code auto mode: A safer way to skip permissions. Anthropic Engineering Blog, March 2026. URL <https://www.anthropic.com/engineering/claude-code-auto-mode>.
- [29] Jaehun Jung, Faeze Brahman, and Yejin Choi. Trust or escalate: LLM judges with provable guarantees for human agreement. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2407.18370>.
- [30] Elliot Myunghoon Kim, Avi Garg, Kenny Peng, and Nikhil Garg. Correlated errors in large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 30038–30066, 2025.
- [31] Malte Kuss and Carl Edward Rasmussen. Assessing approximate inference for binary Gaussian process classification. *Journal of Machine Learning Research*, 6:1679–1704, 2005. URL <https://www.jmlr.org/papers/volume6/kuss05a/kuss05a.pdf>.
- [32] Himabindu Lakkaraju, Jon Kleinberg, Jure Leskovec, Jens Ludwig, and Sendhil Mullainathan. The selective labels problem: Evaluating algorithmic predictions in the presence of unobservables. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 275–284, 2017. doi: 10.1145/3097983.3098066.
- [33] John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004. URL https://doi.org/10.1518/hfes.46.1.50_30392.
- [34] Benjamin Letham, Phillip Guan, Chase Tymms, Eytan Bakshy, and Michael Shvartsman. Look-ahead acquisition functions for Bernoulli level set estimation. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *PMLR*, pages 8493–8513, 2022.
- [35] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/09d37c08f7b129e96277388757530c72-Abstract.html.
- [36] Thomas P. Minka. Expectation propagation for approximate Bayesian inference. In *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 362–369. Morgan Kaufmann, 2001. URL <https://tminka.github.io/papers/ep/minka-ep-uai.pdf>.
- [37] Meredith Ringel Morris, Jascha Sohl-Dickstein, Noah Fiedel, Tris Warkentin, Allan Dafoe, Aleksandra Faust, Clement Farabet, and Shane Legg. Position: Levels of AGI for operationalizing progress on the path to AGI. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 36308–36321. PMLR, 2024. URL <https://proceedings.mlr.press/v235/morris24b.html>.
- [38] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *PMLR*, pages 7076–7087, 2020.
- [39] OpenAI. Codex CLI. Open-source repository, commit ea647275560ffbcd4cf240d048eab44aa66ba9ae, 2026. URL <https://github.com/openai/codex>.

- [40] Christopher J. Paciorek and Mark J. Schervish. Nonstationary covariance functions for Gaussian process regression. In *Advances in Neural Information Processing Systems 16 (NIPS)*. MIT Press, 2003. URL <https://proceedings.neurips.cc/paper/2003/hash/326a8c055c0d04f5b06544665d8bb3ea-Abstract.html>.
- [41] Conner Phillippi. Auto mode is now the default in Claude Code for Pro, Max, and Team plans. Claude Blog, August 2026. URL <https://claude.com/blog/auto-mode-default-in-claude-code>.
- [42] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 978-0-262-18253-9. doi: 10.7551/mitpress/3206.001.0001. URL <https://gaussianprocess.org/gpml/chapters/RW.pdf>.
- [43] Alex Remedios, Simon Storf, Fabien Roger, and John Hughes. Red-teaming auto mode: Improving blocking classifiers against malign coding agents, 2026. URL <https://arxiv.org/abs/2609.19587>.
- [44] Jaakko Riihimäki and Aki Vehtari. Gaussian processes with monotonicity information. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, volume 9 of *PMLR*, pages 645–652, 2010.
- [45] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=GEcwtMk1uA>. Spotlight.
- [46] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009. URL <https://research.cs.wisc.edu/techreports/2009/TR1648.pdf>.
- [47] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016. doi: 10.1109/JPROC.2015.2494218. URL <https://doi.org/10.1109/JPROC.2015.2494218>.
- [48] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O’Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, Katarina Slama, Lama Ahmad, Paul McMillan, Alex Beutel, Alexandre Passos, and David G. Robinson. Practices for governing agentic AI systems. White paper, OpenAI, 2023. URL <https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf>.
- [49] Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. Safe exploration for optimization with Gaussian processes. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *PMLR*, pages 997–1005, 2015.
- [50] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1467–1490. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.79. URL <https://aclanthology.org/2024.findings-emnlp.79/>.

- [51] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of LLM agents, 2024. URL <https://arxiv.org/abs/2412.14470>.
- [52] Tijana Zrnic and Emmanuel Candès. Active statistical inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 62993–63010, 2024. URL <https://proceedings.mlr.press/v235/zrnic24a.html>.